I want to count the number of steps the the user takes. I will be referencing the ECE 16 HW 3 and HW 4 in order to get clean data from the accelerometer. I will also be referencing the ECE 16 syllabus to conceptually understand how this will work.

Live plotting

Circular lists (for live data collection and plotting)

* A circular list here is a data structure in which the last node points back to the first node (forms cont. loop instead of terminating w/ a null pointer)

* In real time data collection, it is used to manage cyclic processes and fixed-sized data buffers where older data is continuously overwritten by newer data

(Circular buffer)
are temp., reserved RAM
storage areas used to hold data
while it is being moved from one place to another
(temp. queue)

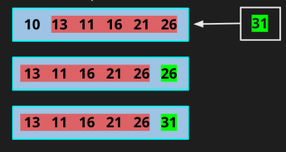
* matplotlib is easy but very slow due to full refresh
* best way (do not use matplotlib) use: Pygame, Visipy, PyQt
* need a FIFO (first in first out) buffer (circular buffer of data)

Circular Lists

A Circular Buffer of Data

What we need for live plotting: a FIFO (First-in-first-out) buffer

- `a = [10, 13, 11, 16, 21, 26]`
- `a.add(31)` *# non-existent function we need!*
- `a = [13, 11, 16, 21, 26, 31]`



We can use slicing!

- `a = [10, 13, 11, 16, 21, 26]`

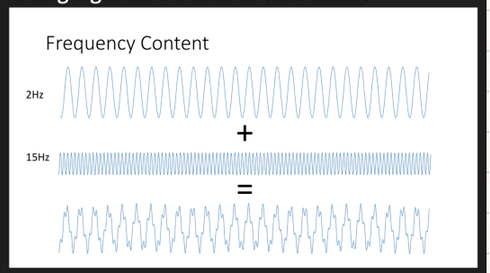
- `a[ :-1] = a[ 1: ]` *# assign a[1..N] to a[0...N-1] (shift data left by 1)*
- `a[ -1 ] = 31` *# assign the new value to the end of the array*

What do we have to do to enable: `a.add([4,5,2,2])`?

DSP (Digital Signal processing)

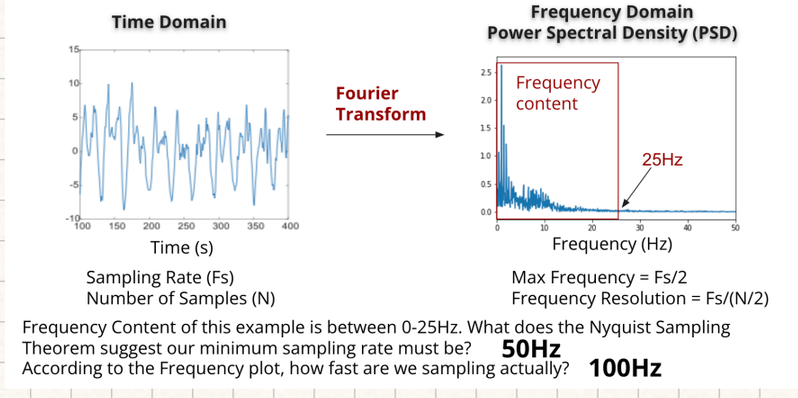
* Using discrete time signal processing? (looks like cont.)
* Common sources of Signal distortion
  - Signal bias
  - noise from electronic components
  - Ambient noise
  - motion artifacts (noise caused by sensor movement during data aquisition)
* goes into Signal filtering → analog and digital
* measuring signals: Sampling theorem → 1) know freq content of signal
  2) Sample at least at Nyquist (2x highest freq.)
  3) In practice, typically Sample at 4x
* remember time and freq. domain (Sec (time) and Hz (freq.))
* measuring signals → use superposition!

Approximating Signals as a Sum of Sine Waves



A signal that varies faster in time has higher frequency components

* Use fourier transform analysis



Sampling Rate (Fs)
Number of Samples (N)

Max Frequency = Fs/2
Frequency Resolution = Fs/(N/2)

Frequency Content of this example is between 0-25Hz. What does the Nyquist Sampling Theorem suggest our minimum sampling rate must be? **50Hz**
According to the Frequency plot, how fast are we sampling actually? **100Hz**

* filter types: low pass, high pass, band pass, and band stop

object-oriented programming

what is oop? → organizing programs into objects rather than functions and variables

- class: the definition
- instance: an entity created using class definition

Class Definition & Construction

```
class Dog:                                  # Class DEFINITION: Define class "Dog"
    name = ""                               # Class ATTRIBUTES: "Dog" Properties
    color = ""                              # - Declaration is optional
    __weight = 0                            # - PRIVATE attribute (encapsulated)
    def __init__(self, name, color, weight):  # CONSTRUCTOR: Initialize "objects"
        self.name = name
        self.color = color
        self.__weight = weight
```

- The definition describes attributes & methods of a "Dog"
- When you make a new "Dog", `__init__()` will be called
- All methods & attributes of a class must have "self" as the first argument

Attributes

- self.name, self.color
  - These attributes are in the scope (accessible) by any of the instance's own methods.
- self.__weight
  - This attribute is encapsulated!
  - print(Spark.__weight) will result in AttributeError: 'Dog' object has no attribute '__weight'

```
def set_bark(self, sound):
    self.__bark = sound

def bark(self):
    print(self.__bark)
```

```
Spark = Dog("Spark", "Brown", 10)
Spark.set_bark("woof!")

Spark.__bark
>> AttributeError
Spark.bark()
>> woof!
```

Methods

- self.set_bark(), self.bark(), self.see_squirrel(), etc.
- Called by dot notation on object instances
- Class methods can call other class methods!

```
def set_bark(self, sound):
    self.__bark = sound
def bark(self):
    print(self.__bark)
def see_squirrel(self):
    self.set_bark("RUFF!!!")
    for i in range(3):
        self.bark()

Scruffy = Dog("Scruff", "White", 2)
Scruffy.see_squirrel()
# Prints "RUFF!!!" 3 times
```

Inheritance vs polymorphism

Inheritance: • focuses on code reuse →
- allows one class to acquire the properties and behaviors (methods / variables) of another class
- establishes parent/child relationship
- allows one class to inherit methods and properties of another class

* uses "extends" keyword to inherit from a class

```
class Animal {
    public function makeSound() {
        return "Some generic animal sound";
    }
}

class Dog extends Animal {
    public function makeSound() {
        return "Bark!";
    }
}

$dog = new Dog();
echo $dog->makeSound();  // Output: Bark!
```

In this example, the `Dog` class inherits from the `Animal` class and overrides the `makeSound` method.

Purpose:

- **Code reuse:** Share common functionality across multiple classes.
- **Organize hierarchy:** Create a logical class structure that reflects real-world relationships.

polymorphism: • means "many forms"
- allows methods (functions) of

objects to take many forms

- one interface or method, multiple implementations

```java
class Animal {
  public void animalSound() {
    System.out.println("The animal makes a sound");
  }
}

class Pig extends Animal {
  public void animalSound() {
    System.out.println("The pig says: wee wee");
  }
}

class Dog extends Animal {
  public void animalSound() {
    System.out.println("The dog says: bow wow");
  }
}
```

```java
class Main {
  public static void main(String[] args) {
    Animal myAnimal = new Animal();  // Create a Animal object
    Animal myPig = new Pig();  // Create a Pig object
    Animal myDog = new Dog();  // Create a Dog object
    myAnimal.animalSound();
    myPig.animalSound();
    myDog.animalSound();
  }
}
```

Key Differences

Feature	Inheritance	Polymorphism
Concept	A mechanism for creating a new class (subclass) based on an existing class (superclass), acquiring its properties and behaviors.	The ability of a single function or object to have many forms or behave differently depending on the context or object type.
Purpose	Promotes code reusability and establishes an "is-a" relationship (e.g., a `Dog` is an `Animal`).	Provides flexibility and allows for generic code that can work with objects of various related classes.
Applied to	Primarily applies to classes, establishing a hierarchy.	Primarily applies to functions or methods (via overloading and overriding).
Types	Single, multi-level, hierarchical, hybrid (note: Java doesn't support multiple inheritance of classes).	Compile-time (method overloading) and run-time (method overriding).

pedometer logic